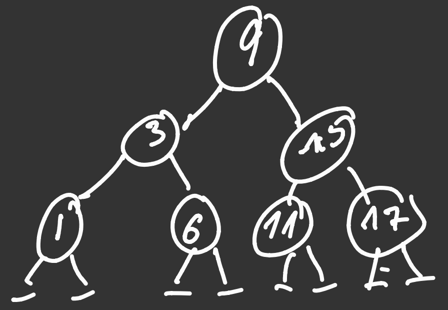
→ Previously

* Isabelle Iface
* Basic reasoning → FW, → BW, → subst, → Induction on nats
* Lists → recursive fn's on lists, → proof by structural induction on lists
* Algebraic datatypes → Nat's, → Lists, → trees

→ This video :- Primitive Recursion on algebraic datatypes

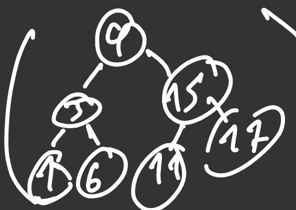
* 

datatype 'a tree
 = Leaf
 | Node "'a tree" "'a" "'a tree"

{1, 3, 6, 9, 11, 15, 17}

* list-set [] = {}
  list-set (x # xs) = insert x (list-set xs)

* tree-set Leaf = {}
  tree-set (Node l a r) = insert a ((tree-set l) ∪ (tree-set r))

* tree-set () = insert 9 (tree-set (3; 1 6) ∪ set (15; 11 17))

= insert 9 (insert 3 ((tree-set (1) ∪ tree-set (6))) ∪ set (15; 11 17))

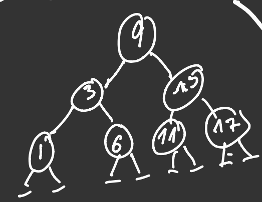
= . . .

→ Inorder traversal of trees

* [1, 3, 6, 9, 11, 15, 17]

* Intuition :- if the tree is a binary search tree (BST), then inorder is a sorted list of elements in the tree
  * BST :- Every node is > all nodes in its left sub-tree
  * " " " < " " " " right sub-tree

* inorder Leaf = []
  inorder (Node l a r) = (inorder l) @ [a] @ (inorder r)

* inorder () = inorder (3; 1 6) @ [9] @ inorder (15; 11 17)

= (inorder (1; ⊥ ⊥) @ [3] @ inorder (6; ⊥ ⊥)) @ [9] @ inorder (13; 11 17)

= (inorder ⊥ @ [1] @ inorder ⊥) @ [3] @ inorder (6) @ [9] @ inorder (13; 11 17)

= ([] @ [1] @ []) @ [3] @ . . .

= [1] @ [3] @ (inorder ⊥ @ [6] @ inorder ⊥) @ [9] @ inorder (15; 11 17)

= [1] @ [3] @ [6] @ . . . . .

* finish this tree on your own !!!